

Plateforme d'Analyse à Interface et Visualisation Graphiques de la Base de Données de la DSA Transparency

Rapport Technique

ABDALLAH Abderraouf, GADEWA Emmanuel, KATTAN Mohamad,
NABET Ibtissam, SUBRUN Shyam

Université Paris-Est Créteil (UPEC)

Résumé

Ce rapport technique présente le développement d'une plateforme d'analyse à interface et visualisation graphiques de la base de données de la DSA transparency, base de données réglementaire contenant plus de 11 milliards de décisions de modération des très grandes plateformes de réseaux sociaux en ligne européennes. Notre solution propose une architecture trois-tiers moderne (React/Express/PostgreSQL) permettant aux chercheurs en droit du numérique d'explorer ces données massives sans compétences en programmation. Nous analysons les défis liés aux paramètres 3V des mégadonnées [25] (Volume, Variété, Vitesse), présentons l'intégration de composants d'intelligence artificielle pour la prédiction des décisions de modération (97 % de précision), et documentons les performances mesurées. Un module innovant à base d'apprentissage automatique permet de prédire automatiquement le type de décision de modération à partir des caractéristiques d'une entrée, et un module de traitement automatique du langage naturel permet de formuler des requêtes analytiques sans aucune connaissance en SQL. Notre outil est utilisable sans aucune formation et permet de répondre à plusieurs cas d'utilisation concrets : chercheurs en droit du numérique, régulateurs, journalistes d'investigation, et analystes de conformité. Le système offre huit sections d'analyse interactive, des filtres dynamiques multi-critères, et des visualisations exportables conformes aux exigences du Digital Services Act.

Mots-clés : DSA, Digital Services Act, mégadonnées, modération de contenu, visualisation de données, apprentissage automatique, React, PostgreSQL

I. Introduction

Le code source de la plateforme développée dans le cadre de ce projet est accessible publiquement via un dépôt GitHub. Cette mise à disposition vise à garantir la transparence des choix techniques, la reproductibilité des résultats ainsi que la possibilité de réutilisation et d'extension du système par d'autres utilisateurs ou chercheurs.

Le projet est disponible à l'adresse suivante : <https://github.com/raouf-rak/dsa-dashboard>.

A. Contexte réglementaire

Le *Digital Services Act* (DSA) de l'Union Européenne, entré en vigueur en 2024, impose aux très grandes plateformes de réseaux sociaux en ligne (VLOPs – *Very Large Online Platforms*) une obligation de transparence sans précédent concernant leurs pratiques de modération de contenu [1]. La **modération de contenu** désigne l'ensemble des processus par lesquels une plateforme numérique détecte, examine et prend des mesures à l'égard de contenus publiés par ses utilisateurs – qu'il s'agisse de supprimer une publication, de restreindre un compte, de retirer une vidéo, ou d'apposer un avertissement – en application de la loi ou de ses propres conditions d'utilisation. Le terme **plateforme** désigne ici exclusivement des plateformes de réseaux sociaux et de diffusion de contenu visées par le DSA, et non les plateformes d'infrastructure fonuagique. Chaque décision de modération doit être

documentée via un *Statement of Reasons* (SoR), alimentant la base de données de la DSA transparency accessible publiquement.

Cette base de données, hébergée par la Commission Européenne à l'adresse <https://transparency.dsa.ec.europa.eu/>, agrège actuellement plus de **11 milliards** de décisions de modération provenant de plateformes numériques incluant réseaux sociaux et places de marché telles que Meta (Facebook, Instagram), TikTok, YouTube, X (anciennement Twitter), LinkedIn, et Amazon. Les décisions de modération sont appliquées par les plateformes elles-mêmes, sous le contrôle et la supervision des autorités compétentes désignées par les États membres de l'Union Européenne – et en France, par l'Autorité de Régulation de la Communication Audiovisuelle et Numérique (ARCOM).

B. Problématique

Depuis 2024, les grandes plateformes numériques de réseaux sociaux (Facebook, TikTok, YouTube, etc.) sont tenues par la loi européenne de justifier publiquement chaque décision de modération de contenu. Ces justifications sont centralisées dans une base de données publique contenant plus de 11 milliards d'entrées.

Le problème est que cette masse de données est difficilement exploitable en l'état. Un juriste, un chercheur en sciences politiques ou un journaliste souhaitant analyser les pratiques de modération se heurte à plusieurs

obstacles :

- **Volume considérable** : plusieurs millions de nouvelles entrées chaque jour, représentant environ 22 Go de données quotidiennes une fois décompressées. À titre de comparaison, cela équivaut à environ 11 millions de pages de texte par jour.
- **Données hétérogènes** : chaque plateforme utilise ses propres conventions pour nommer les catégories de violation, les types de décision et les bases légales invoquées.
- **Mises à jour continues** : les données sont actualisées quotidiennement avec une fenêtre glissante de 6 mois.
- **Absence d’outil d’analyse accessible** : l’exploitation de ces données nécessite actuellement des compétences en programmation (Python, SQL), ce qui exclut de fait la majorité des utilisateurs cibles – juristes, chercheurs en droit du numérique, régulateurs, journalistes.

Notre projet vise à combler ce fossé en offrant une interface visuelle permettant à tout utilisateur, sans compétence technique, d’explorer, filtrer et analyser ces données de modération.

C. Objectifs du projet

Notre projet vise à développer un outil d’analyse « grand public » répondant aux besoins spécifiques des chercheurs en droit du numérique, avec les objectifs suivants :

1. Une interface graphique web intuitive permettant l’exploration sans programmation
2. Des visualisations interactives : tableaux de bord, cartes géographiques, séries temporelles
3. Des analyses exportables aux formats PNG, CSV et PDF
4. Une intégration de composants d’intelligence artificielle pour l’analyse prédictive
5. Une mesure des performances et de l’extensibilité

Le reste du document est organisé comme suit : la section II présente l’analyse des données et le contexte juridique, la section III détaille l’architecture technique, la section IV décrit le schéma de données, la section V présente les fonctionnalités de l’interface, la section VI couvre l’intégration de l’apprentissage automatique et du TALN, la section VII détaille les cas d’usage, la section VIII analyse les performances et les paramètres 3V, la section IX traite les considérations RGPD. La conclusion, les perspectives ainsi que la référence de l’API sont présentées en appendice.

II. Analyse des Données DSA

A. Structure des données source

La base de données de la DSA transparency fournit des données via une interface de programmation (API) REST au format JSON. Chaque entrée correspond à une décision de modération unique (un *Statement of Reasons*) et

contient les champs présentés dans le Tableau 1. L’identifiant unique (*uuid*) permet de tracer chaque décision individuellement à travers le système.

TABLE 1 – Structure des données DSA principales

Champ	Description et exemples
<i>uuid</i>	Identifiant unique universel (UUID v4). Ex : 550e8400-e29b-41d4...
<i>platform_name</i>	Plateforme de réseau social. Ex : TikTok, Facebook, YouTube
<i>application_date</i>	Date d’application au format ISO 8601. Ex : 2025-01-15
<i>content_date</i>	Date de création du contenu modéré. Ex : 2024-12-20
<i>category</i>	Catégorie de violation. Ex : ILLEGAL_OR_HARMFUL_SPEECH, VIOLENCE
<i>decision_ground</i>	Base légale ou contractuelle. Ex : <i>Illegal content (Art. 3 DSA)</i>
<i>decision_visibility</i>	Type de restriction. Ex : REMOVAL, DISABLED
<i>territorial_scope</i>	Codes pays ISO 3166-1 alpha-2 [5]. Ex : ["FR", "DE"]
<i>automated_detection</i>	Détection par algorithme (<i>true</i>) ou humain (<i>false</i>)
<i>automated_decision</i>	Niveau d’automatisation. Ex : FULLY_AUTOMATED, NOT_AUTOMATED
<i>content_type</i>	Format du contenu. Ex : TEXT, IMAGE, VIDEO, AUDIO
<i>content_language</i>	Code langue ISO 639-1 [6]. Ex : fr, de, en, el

Détails sur les identifiants UUID. L’identifiant *uuid* suit le format UUID version 4 défini par la RFC 4122 [7]. Il s’agit d’une chaîne de 36 caractères (dont 4 tirets) au format `xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx`, où *x* est un chiffre hexadécimal et *y* vaut 8, 9, a ou b. Le chiffre 4 en troisième groupe indique la version actuellement en cours d’utilisation (version 4), qui génère les identifiants par tirage pseudo-aléatoire – contrairement aux versions 1 et 2 qui utilisaient l’horodatage ou l’identifiant de la machine. Les sous-chaînes représentent des champs historiques : *time_low* (8 car.), *time_mid* (4 car.), *time_hi_and_version* (4 car., dont le premier *nibble* encode la version), *clock_seq* (4 car.) et *node* (12 car.). En version 4, tous ces champs sont remplis pseudo-aléatoirement sauf les bits de version et de variante, garantissant l’unicité statistique sans coordination centralisée. La probabilité de collision entre deux UUID v4 est de l’ordre de 5×10^{-36} pour un milliard d’identifiants générés.

Codes ISO. Le champ *territorial_scope* utilise la norme ISO 3166-1 alpha-2 [5], qui attribue un code de deux lettres majuscules à chaque pays (FR pour la France, DE pour l’Allemagne). Le champ *content_language* utilise la norme ISO 639-1 [6], qui attribue un code de deux lettres minuscules à chaque langue (*fr* pour le français, *en* pour l’anglais).

Champ *automated_detection*. Ce champ booléen indique si le contenu problématique a été identifié par

un algorithme automatisé (filtrage par empreinte numérique, classificateur d'apprentissage automatique, etc.) ou par un signalement humain (utilisateur, signaleur de confiance, autorité compétente). En pratique, les grandes plateformes de réseaux sociaux détectent automatiquement entre 60 et 90 % des contenus modérés [2]. Cette distinction renseigne sur le degré de dépendance des plateformes vis-à-vis de leurs systèmes automatisés.

Champ `automated_decision`. Ce champ à trois valeurs indique le niveau d'intervention humaine dans la prise de décision. `FULLY_AUTOMATED` signifie qu'aucun opérateur humain n'a examiné le contenu avant l'application de la sanction ; `PARTIALLY_AUTOMATED` indique qu'un algorithme a proposé une action ensuite validée ou modifiée par un opérateur ; `NOT_AUTOMATED` signifie que la décision a été prise entièrement par un humain. L'article 22 du RGPD [4] garantit aux utilisateurs un droit à l'explication et à la contestation des décisions entièrement automatisées.

B. Concepts juridiques clés

Cette sous-section explicite les concepts juridiques du DSA qui se traduisent directement en attributs techniques dans notre base de données. La compréhension de ces distinctions est essentielle pour interpréter correctement les champs `decision_ground`, `automated_detection` et `automated_decision`.

1) Contenu illégal vs incompatible

Le DSA (Article 3, paragraphe h [1]) établit une distinction fondamentale entre deux types de contenu problématique, chacun correspondant à une valeur distincte du champ `decision_ground` :

- **Contenu illégal** (`ILLEGAL_CONTENT`) : contenu qui viole une loi européenne ou nationale (ex : incitation à la haine raciale au sens de la Directive 2011/93/UE [3], vente de produits contrefaits, matériel pédopornographique). Le fondement est juridique et étatique. La plateforme engage sa responsabilité légale si elle n'agit pas après signalement (Article 6 du DSA [1]).
- **Contenu incompatible** (`INCOMPATIBLE_CONTENT`) : contenu qui enfreint les conditions générales d'utilisation (CGU) de la plateforme sans être illégal au sens du droit (ex : pourriel, désinformation, contenu explicite sur une plateforme tout public). Le fondement est contractuel. La responsabilité de la plateforme est moindre mais le DSA impose la transparence des critères appliqués (Article 14 [1]).

En termes d'implémentation, cette distinction se matérialise dans le champ `decision_ground` et influence directement les visualisations de la section *Bases juridiques* (cf. Figure 15).

2) Automatisation et responsabilité

Le DSA (Articles 14 et 15 [1]) encadre strictement l'utilisation d'outils automatisés dans le processus de modération. La **détection automatisée** (`automated_detection`) indique si le contenu a été identifié par un algorithme ou par un signalement humain. La **décision automatisée** (`automated_decision`) indique si l'action de modération a été prise sans intervention humaine, selon trois niveaux : `FULLY_AUTOMATED`, `PARTIALLY_AUTOMATED`, et `NOT_AUTOMATED`. Ces deux dimensions sont croisées dans la section *Automatisation* du tableau de bord (carte thermique automatisation × catégorie × plateforme).

III. Architecture Technique

A. Vue d'ensemble

Notre plateforme adopte une architecture trois-tiers moderne, complétée par un module dédié à l'intelligence artificielle (traitement automatique du langage naturel (TALN) et apprentissage automatique), illustrée dans la Figure 1. Chaque couche joue un rôle fonctionnel précis : la **couche présentation** (React/Vite) gère l'interface utilisateur et la visualisation interactive des données ; le **serveur applicatif** (Express.js/Prisma) orchestre la logique métier, le filtrage, l'agrégation des requêtes et la communication avec la base de données ; le **module IA/-TALN** (FastAPI [16]/Python) interprète les requêtes en langage naturel, génère des requêtes SQL et fournit les prédictions de modération ; enfin, la **base de données** PostgreSQL centralise l'ensemble des décisions de modération et des métadonnées.

Il convient de rappeler que les décisions de modération sont **appliquées par les plateformes elles-mêmes** (TikTok, Meta, YouTube, etc.) : la plateforme développée dans ce projet n'applique aucune sanction, elle sert uniquement à **analyser** et **visualiser** les décisions déjà publiées.

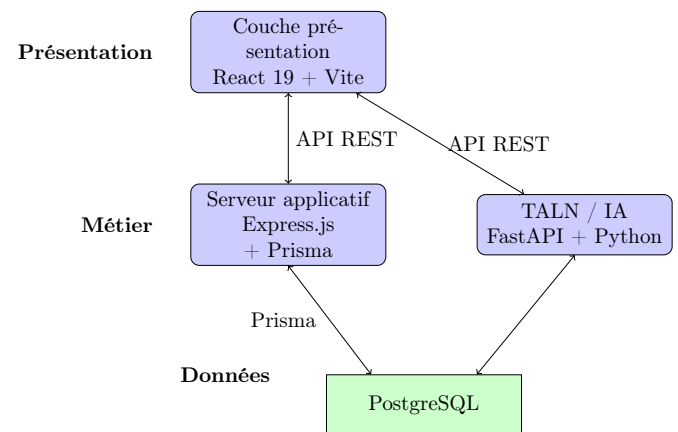


FIGURE 1 – Architecture trois-tiers avec base de données partagée et communication via FastAPI

B. Pile technologique

1) Couche présentation (côté client)

- **React 19** [8] : bibliothèque d'interface utilisateur avec *hooks* et composants fonctionnels
- **TypeScript** : typage statique pour la robustesse du code
- **Vite 7** [13] : empaqueteur moderne pour le développement rapide
- **TanStack Query** [14] : gestion du cache et des requêtes asynchrones
- **ECharts 6** [9] : bibliothèque de visualisation interactive

2) Serveur applicatif (côté serveur)

- **Node.js** [18] : environnement d'exécution JavaScript côté serveur
- **Express.js** [19] : cadriciel web minimaliste
- **Prisma ORM** [10] : correspondance objet-relationnelle avec typage sûr
- **TypeScript** : typage de bout en bout

3) Base de données – Choix et justification

Le choix du SGBD a été guidé par une étude comparative de trois candidats : MySQL [20], SQLite [21] et PostgreSQL [11]. **Décision technique** : PostgreSQL version 17 a été retenu pour sa conformité ACID, son support natif des types avancés (JSONB, tableaux, UUID), sa richesse analytique (expressions de table communes récursives, fonctions de fenêtrage, agrégations avancées) et son excellente intégration Python. Les travaux de Juba et Vannahme [26] confirment ses avantages pour les charges analytiques sur gros volumes.

Les fonctionnalités exploitées incluent les **vues matérialisées** pour le pré-calcul des agrégations des tableaux de bord, les **index B-tree** pour accélérer les filtres fréquents, et les **déclencheurs** (*triggers*) pour la synchronisation automatique des tables de référence.

4) Infrastructure et hébergement infonuagique

Décision technique : Google Cloud Platform [24] a été retenu pour sa simplicité opérationnelle, son intégration IA native (API Gemini [15]), sa latence optimisée pour PostgreSQL et son coût maîtrisé. Une instance Compute Engine de série N2 a été provisionnée (4 processeurs virtuels, 16 Go de mémoire vive, SSD 150 Go). L'ensemble des services applicatifs est orchestré via Docker Compose [17] avec Nginx comme mandataire inverse.

Le choix de GCP a été préféré aux alternatives AWS [22] et Azure [23] notamment pour son intégration native avec l'API Gemini, déterminante pour le module TALN.

Dépendances logicielles et considérations RGPD. Ces choix technologiques introduisent des dépendances vis-à-vis de tiers et soulèvent des questions de conformité au RGPD [4]. En particulier, l'utilisation de l'API Gemini

(Google) implique que les requêtes formulées par les utilisateurs en langage naturel sont transmises aux serveurs de Google pour traitement. Si un utilisateur analyse un cas impliquant une personne physique identifiable, ces données pourraient transiter vers des serveurs hors Union Européenne, constituant un transfert soumis aux articles 44 à 49 du RGPD. Des mesures d'atténuation possibles incluent l'anonymisation systématique des entrées avant envoi à l'API, ou l'utilisation de modèles de langage auto-hébergés. Dans l'état actuel du prototype académique, cette limitation est reconnue et documentée ; une version de production devrait faire l'objet d'une analyse d'impact (AIPD) conformément à l'article 35 du RGPD.

5) Conteneurisation des services

La **conteneurisation** via Docker apporte trois avantages majeurs. Premièrement, la **portabilité** : le même service s'exécute identiquement en développement, test ou production. Deuxièmement, l'**isolation des dépendances** : chaque composant (frontend, backend, IA) embarque ses propres bibliothèques, évitant les conflits logiciels. Troisièmement, la **reproductibilité du déploiement** : une image construite une fois peut être redéployée sans divergence d'environnement. Dans notre architecture, le frontend, le backend et le module IA sont encapsulés dans des conteneurs distincts, orchestrés par Docker Compose.

C. Organisation du code

Listing 1 – Structure du projet

```
dsa-dashboard/
|- backend/
|  |- src/
|  |  |- controllers/      # Logique HTTP
|  |  |- routes/          # Points d'accès API
|  |  |- services/        # Logique métier
|  |  |- config/          # Configuration
|  |  +-- types/          # Types TypeScript
|  |  |- prisma/          # Schema ORM
|  +-- Dockerfile
|- frontend/
|  |- src/
|  |  |- components/      # Composants React
|  |  |- hooks/           # Hooks personnalisés
|  |  |- context/         # Contextes React
|  |  |- data/            # Services API
|  |  |- utils/           # Utilitaires
|  |  +-- styles/         # CSS globaux
|  +-- Dockerfile
|- database/              # Scripts SQL
|- monitoring/            # Observabilité
+-- docker-compose.yml
```

IV. Schéma de Données

A. Modèle relationnel normalisé

Le schéma de données a été conçu selon une logique de normalisation afin de réduire la redondance, d'améliorer la cohérence et d'optimiser les performances des requêtes analytiques. Les valeurs fréquemment répétées – noms de plateformes, catégories, types de décision – sont isolées dans des tables de référence. La table principale ne stocke que des identifiants numériques courts

(SMALLINT) plutôt que des chaînes de caractères, ce qui réduit considérablement sa taille sur des dizaines de millions de lignes. La table centrale `moderation_entries` stocke les faits de modération, tandis que les tables `platforms`, `categories`, `decision_types`, `decision_grounds` et `content_types` jouent le rôle de dimensions. Les déclencheurs (*triggers*) PostgreSQL assurent la mise à jour automatique de ces tables de référence lors de l'insertion de nouvelles entrées. Les vues matérialisées pré-calculent les agrégations les plus fréquentes (comptages par plateforme, catégorie, pays) pour réduire les temps de réponse à quelques dizaines de millisecondes.

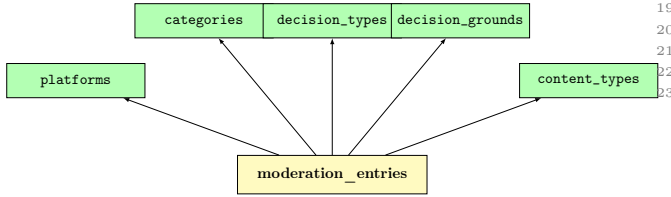


FIGURE 2 – Modèle entité-relation simplifié

B. Tables de référence

Les tables de référence contiennent les valeurs normalisées partagées par de nombreuses entrées. Une plateforme comme TikTok n'est stockée qu'une seule fois dans `platforms`, puis référencée par clé étrangère. Cette organisation diminue la taille totale de la base, réduit le risque d'incohérences textuelles et rend les jointures plus efficaces.

Listing 2 – Définition des tables de référence

```

1 CREATE TABLE platforms (
2   id SMALLSERIAL PRIMARY KEY,
3   name VARCHAR(100) NOT NULL UNIQUE,
4   created_at TIMESTAMP DEFAULT NOW()
5 );
6 CREATE TABLE categories (
7   id SMALLSERIAL PRIMARY KEY,
8   name VARCHAR(100) NOT NULL UNIQUE,
9   created_at TIMESTAMP DEFAULT NOW()
10 );
11 CREATE TABLE decision_types (
12   id SMALLSERIAL PRIMARY KEY,
13   name VARCHAR(100) NOT NULL UNIQUE,
14   created_at TIMESTAMP DEFAULT NOW()
15 );
16 CREATE TABLE decision_grounds (
17   id SMALLSERIAL PRIMARY KEY,
18   name TEXT NOT NULL UNIQUE,
19   created_at TIMESTAMP DEFAULT NOW()
20 );
21 CREATE TABLE content_types (
22   id SMALLSERIAL PRIMARY KEY,
23   name VARCHAR(50) NOT NULL UNIQUE,
24   created_at TIMESTAMP DEFAULT NOW()
25 );

```

C. Table principale

La table principale centralise chaque décision de modération avec sa date, son territoire, sa langue, son niveau d'automatisation et ses références vers les dimensions. Elle constitue la table de faits exploitée par les filtres, les visualisations et les modules d'analyse.

Listing 3 – Table principale `moderation_entries`

```

1 CREATE TABLE moderation_entries (
2   id VARCHAR(50) PRIMARY KEY,
3   application_date DATE NOT NULL,
4   content_date DATE,
5   platform_id SMALLINT NOT NULL
6     REFERENCES platforms(id),
7   category_id SMALLINT NOT NULL
8     REFERENCES categories(id),
9   decision_type_id SMALLINT NOT NULL
10    REFERENCES decision_types(id),
11   decision_ground_id SMALLINT NOT NULL
12    REFERENCES decision_grounds(id),
13   incompatible_content_ground TEXT,
14   content_type_id SMALLINT
15    REFERENCES content_types(id),
16   automated_detection BOOLEAN,
17   automated_decision VARCHAR(30),
18   country_code CHAR(2),
19   territorial_scope JSONB,
20   language VARCHAR(10),
21   delay_days INTEGER,
22   created_at TIMESTAMP DEFAULT NOW()
23 );

```

Le champ `automated_decision` est stocké sous forme textuelle afin de conserver les trois modalités fonctionnelles de la base DSA (`FULLY_AUTOMATED`, `PARTIALLY_AUTOMATED`, `NOT_AUTOMATED`). Le choix de `JSONB` pour `territorial_scope` permet de représenter des portées territoriales multiples dans un format semi-structuré, compatible avec les besoins d'analyse géographique.

V. Interface Utilisateur et Fonctionnalités

L'interface du tableau de bord de la DSA Transparency est présentée dans les Figures 11 à 19. Notre approche de visualisation se distingue d'une solution BI généraliste (Tableau, Power BI) par son intégration native du vocabulaire réglementaire du DSA (bases légales, catégories de violation, automatisation des décisions), son module de requêtes en langage naturel couplé à la navigation, et son déploiement sans installation côté client. Ces choix sont particulièrement adaptés aux profils non informaticiens visés par le projet : juristes, journalistes, régulateurs et analystes de conformité.

A. Sections d'analyse

1) Custom Query Chart

Le tableau de bord intègre, en tête de page, un module *Custom Query Chart* permettant à l'utilisateur de décrire en langage naturel le type de visualisation souhaité. Ce composant constitue le point d'entrée principal pour l'exploitation assistée par TALN. *Cas d'usage typique (CU1, CU2, CU3, CU4)* : un utilisateur formule directement une question analytique sans connaître la structure de la base ni le langage SQL.

2) Vue d'ensemble et séries temporelles (Figure 11)

Cette section présente les indicateurs synthétiques principaux : nombre total d'actions (17 millions), nombre de plateformes actives (15), délai moyen de traitement (51,8 jours), taux de détection automatisée (61,9 %), taux

de décision automatisée (52 %) et nombre de pays couverts (27). Elle est complétée par des séries temporelles montrant l'évolution mensuelle du volume d'actions et du délai moyen de réponse. *Cas d'usage typique (CU2 – Régulateur)* : obtenir un état des lieux global avant d'approfondir l'analyse par plateforme ou catégorie.

3) Évolution mensuelle par plateforme (Figure 12)

Cette section présente l'évolution mensuelle des actions de modération pour chacune des 15 plateformes couvertes, de septembre 2025 à avril 2026. La vue en séries temporelles multi-courbes permet de comparer visuellement les tendances d'activité, d'identifier des pics saisonniers et de repérer des comportements atypiques. *Cas d'usage typique (CU1 – Chercheur)* : après avoir sélectionné TikTok dans les filtres, le chercheur isole la courbe TikTok pour observer les pics d'activité mensuels et les corréliser avec des événements réglementaires ou médiatiques.

4) Comparaison des plateformes (Figure 13)

Cette section compare les plateformes selon trois angles complémentaires : volume total d'actions (histogramme horizontal), répartition des types de décision par plateforme (diagramme empilé), et profil catégoriel des trois principales plateformes – TikTok, YouTube et Instagram – sous forme de diagramme radar. La combinaison de ces trois vues permet une lecture comparative rapide des politiques de modération. *Cas d'usage typique (CU1 – Chercheur)* : comparer les catégories de violation dominantes entre TikTok, YouTube et Instagram.

5) Bases juridiques (Figure 15)

Cette section met en évidence les dix principaux fondements juridiques ou contractuels invoqués pour justifier les décisions de modération. Les motifs les plus fréquemment mobilisés sont les violations des conditions d'utilisation (*Terms of Service violation*, 3 391 330 actions), les contenus illégaux au sens de l'article 3 du DSA (*Illegal content*, 3 115 335 actions), les règles communautaires de la plateforme (*Platform community guidelines*, 1 975 655 actions), ainsi que les contenus nuisibles non illégaux (*Harmful but legal content*, 1 685 975 actions). Viennent ensuite les transpositions nationales du DSA, la propriété intellectuelle, la Directive 2011/93/UE relative à la protection de l'enfance, les violations du RGPD, le droit de la consommation, et le Règlement 2021/784 relatif à la diffusion de contenus terroristes. *Cas d'usage typique (CU3 – Journaliste)* : repérer les motifs dominants dans un contexte national donné et exporter ce graphique pour une publication.

6) Répartition géographique (Figure 16)

La carte choroplèthe représente la distribution territoriale des actions de modération à l'échelle des 27 États membres de l'Union européenne. L'intensité de la couleur bleue encode le volume d'actions par pays, selon une échelle allant de 0 à 2 502 275 actions. Les pays à fort volume – notamment la France, l'Allemagne et la Pologne

– apparaissent en bleu foncé, tandis que les pays à plus faible activité de modération restent en teintes claires. *Cas d'usage typique (CU4 – Analyste)* : identifier rapidement les disparités entre États membres et orienter une analyse comparative transnationale.

7) Type de contenu (Figure 19)

Cette section décrit la modération selon le format du contenu selon deux angles complémentaires. Le premier graphique (histogramme empilé) compare les types de décision – suppression, suspension de compte, restriction de visibilité, avertissement, démonétisation – selon les six formats de contenu : image, texte, vidéo, diffusion en direct (*live stream*), audio et story/reel. Le second graphique (diagramme en anneau) représente la distribution globale des formats : image (27,64 %), texte (24,94 %), vidéo (22,42 %), diffusion en direct (11,25 %), audio (8,72 %) et story/reel (5,03 %). *Cas d'usage typique (CU2 – Régulateur)* : analyser si certains formats concentrent davantage certaines sanctions, ou si des formats émergents comme la diffusion en direct font l'objet d'un traitement spécifique.

8) Système de filtres (Figure 21)

Les filtres multi-critères sont accessibles via le panneau latéral dédié. Ils permettent de restreindre l'analyse selon la période, la plateforme, la catégorie, le type de décision, la base juridique, le pays, le type de contenu et le degré d'automatisation. Cette logique de filtrage global permet d'appliquer instantanément les mêmes contraintes à l'ensemble des visualisations.

Listing 4 – Modèle de l'état des filtres

```
interface FilterState {
  1  dateRange: { start: string; end: string } | null;
  2  platforms: string[];
  3  categories: string[];
  4  decisionTypes: string[];
  5  decisionGrounds: string[];
  6  countries: string[];
  7  contentTypes: string[];
  8  automatedDetection: boolean | null;
  9  automatedDecision: boolean | null;
  10 }
  11 }
```

Les filtres sont gérés via un contexte React pour assurer une propagation cohérente à tous les composants, et les requêtes sont optimisées par TanStack Query avec une mémoire cache de 5 minutes.

VI. Apprentissage Automatique et TALN

A. Problématique de prédiction

Notre modèle d'apprentissage automatique vise à prédire automatiquement la décision de modération à partir des caractéristiques de l'entrée, permettant l'identification des incohérences dans les décisions historiques, la détection des cas ambigus nécessitant révision humaine, et l'audit des pratiques de modération.

B. Pipeline de données

Notre chaîne de traitement suit un enchaînement explicite :

**ZIP bruts → filtrage → alignement → import
PostgreSQL → préparation ML**

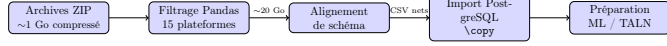


FIGURE 3 – Pipeline de traitement des données DSA depuis les archives brutes jusqu’aux modules analytiques.

Le script de filtrage traite les données DSA brutes sous forme d’archives ZIP imbriquées (environ 1 Go compressé, jusqu’à 20 Go décompressé) avec une lecture par blocs de 200 000 lignes via Pandas afin d’éviter la saturation mémoire. Le filtrage cible 15 plateformes stratégiques : TikTok, Instagram, Facebook, YouTube, X, Snapchat, Reddit, LinkedIn, Amazon Store, AliExpress, Temu, Shein, XVideos, Pornhub et XNXX. Cette sélection réduit le volume de données de 85–90 %.

L’import utilise la commande native PostgreSQL `\copy` via `psql`, qui offre des performances nettement supérieures aux méthodes Python classiques pour l’ingestion massive.

Il est important de préciser que notre système n’est **pas un système temps réel**. Il s’agit d’une **preuve de concept statique ou quasi statique**, fondée sur des extractions périodiques de la base DSA publique. Ce choix est volontairement adapté au cadre académique du projet et reste suffisant pour valider les capacités d’exploration, de visualisation et de prédiction.

C. Modèle de classification

Le modèle vise à prédire automatiquement le type de décision prise par les plateformes (restriction de visibilité, suspension de compte, sanction monétaire ou suppression). Les caractéristiques utilisées sont : le motif de décision (`decision_ground`), la catégorie de contenu, le type de contenu, le nom de la plateforme, la détection automatisée, le type de source, et la langue du contenu. Nous utilisons une régression logistique multiclasse via scikit-learn [12] avec parallélisation activée.

D. Résultats

TABLE 2 – Performances du modèle de classification

Métrique	Valeur
Exactitude globale (<i>accuracy</i>)	97 %
Précision pondérée (<i>weighted precision</i>)	97 %
Rappel pondéré (<i>weighted recall</i>)	97 %
Score F1 pondéré	97 %
Nombre de classes	10
Taille jeu d’entraînement	160 000 lignes (80 %)
Taille jeu de test	40 000 lignes (20 %)

Les classes rares restent difficiles à prédire, ce qui reflète la complexité réelle des cas limites.

E. Stockage et audit

Les prédictions sont persistées dans une table dédiée `dsa_train_test` avec : la décision réelle comparée à la décision prédite, le score de confiance, un indicateur d’erreur, et la version du modèle.

F. Intégration TALN

Le module TALN permet aux utilisateurs d’exprimer une question analytique ou un scénario de modération en langage courant, sans connaissance préalable du schéma de données ni du langage SQL. Le système extrait automatiquement les valeurs distinctes des colonnes stratégiques depuis PostgreSQL, construit une invite structurée, puis s’appuie sur Gemini [15], utilisé via une API distante pour interpréter la demande et produire une représentation exploitable.

Le pipeline fonctionne en trois étapes. **Étape 1 – Extraction du référentiel** : le script extrait toutes les valeurs distinctes de sept colonnes stratégiques (`decision_ground`, `category`, `content_type`, `platform_name`, `automated_detection`, `source_type`, `content_language`) et les exporte dans un fichier JSON (`dsa_reference_values.json`) servant de dictionnaire de référence, garantissant la synchronisation avec les données réelles de la base. **Étape 2 – Génération de l’invite Gemini** : une invite structurée est construite avec tous les attributs possibles indexés numériquement (ex : 0 = `ILLEGAL_CONTENT`, 1 = `INCOMPATIBLE_CONTENT`). L’invite instruit Gemini à répondre uniquement avec des indices numériques, le cadrant strictement pour éviter toute interprétation arbitraire. **Étape 3 – Pipeline de bout en bout** : l’utilisateur saisit une phrase en langage naturel ; Gemini retourne une séquence d’indices décodée via le référentiel pour reconstituer les caractéristiques du modèle d’apprentissage automatique ; le pipeline scikit-learn produit une prédiction finale avec son niveau de confiance et les 10 caractéristiques ayant le plus contribué à la décision (explicabilité via les coefficients de régression logistique).

Exemple concret : « *vente d’armes sur LinkedIn* »
→ {category : `UNSAFE_PRODUCTS`, platform : `LinkedIn`}
→ Prédiction : `REMOVAL`.

Cette architecture modulaire en trois étapes présente des avantages concrets. La **maintenabilité** : si la base DSA évolue, il suffit de ré-exécuter les scripts d’extraction et de génération d’invite pour mettre à jour automatiquement l’ensemble du pipeline. La **séparation des responsabilités** : Gemini joue le rôle d’analyste sémantique, tandis que la régression logistique assure la classification statistique robuste. La **transparence** : en forçant Gemini à répondre par des indices numériques, chaque étape est auditable, ce qui permet de répondre aux exigences de transparence des systèmes automatisés imposées par le DSA.

G. Génération SQL via Gemini

Un second module permet de transformer une question en langage naturel en requête SQL Post-

greSQL directement exécutable. Le script interroge `information_schema.columns` afin de récupérer automatiquement le schéma de la table `moderation_entries`. L'invite impose des contraintes fortes : produire uniquement du SQL PostgreSQL valide, cibler exclusivement la table `moderation_entries`, et retourner une requête compatible avec les visualisations. Des règles métier spécifiques sont intégrées (ex : l'opérateur `LIKE` pour les filtres géographiques sur `territorial_scope`).

En termes de qualité, nos tests montrent que le module génère correctement l'ensemble des constructions SQL de base : `SELECT`, `WHERE`, `GROUP BY`, `ORDER BY`, `HAVING`, `JOIN`, sous-requêtes et agrégations (`COUNT`, `AVG`, `SUM`). Les cas complexes impliquant des requêtes multi-intentions restent un axe d'amélioration. En production, la génération SQL en temps réel via Gemini requiert environ 200 ms par requête et consomme ~500 tokens par appel API ; une montée en charge à 100 utilisateurs simultanés nécessiterait environ 2 instances du module IA en parallèle.

H. Génération de visualisations en langage naturel

Le tableau de bord intègre un module *Custom Query Chart* (Figure 22) permettant à l'utilisateur de décrire en langage naturel le type d'analyse souhaité. Le fonctionnement suit quatre étapes : (1) saisie de la question dans une zone de texte avec des exemples prédéfinis pour guider les utilisateurs novices ; (2) interprétation par Gemini en spécification analytique (dimensions, métriques, filtres) ; (3) exécution de la requête sur la base de données ; (4) rendu automatique d'une visualisation interactive (histogramme, série temporelle, carte thermique). Une ligne de métadonnées indique le nombre de lignes agrégées et le temps de réponse, renforçant la transparence sur le coût de la chaîne *TALN* → *SQL* → *visualisation*.

I. Génération automatisée de rapports analytiques

Au-delà de la génération de graphiques, la plateforme intègre une chaîne complète de production de rapports analytiques à partir d'une requête en langage naturel. La chaîne peut être résumée ainsi :

TALN → SQL → Données → Analyse

Concrètement : la requête en langage naturel est traduite en SQL ; la requête SQL est vérifiée et normalisée ; les données sont extraites depuis PostgreSQL ; les résultats sont transmis à un modèle d'IA générative pour produire un rapport analytique synthétique ; le rapport est automatiquement sauvegardé dans un fichier texte pour exploitation ou archivage. Cette fonctionnalité facilite l'accès à l'analyse pour des profils non techniques en mettant en place une chaîne complète et automatisée.

J. Extension comparative multi-pays

Une extension naturelle consiste à interroger le module TALN sur une même action de modération appliquée à

plusieurs pays afin d'effectuer des comparaisons transnationales. L'utilisateur peut demander, par exemple, de comparer la modération d'un même type de contenu entre la France, l'Allemagne et un troisième pays européen. Une telle requête permet d'analyser les différences de durée moyenne de prise de décision, les variations de types de restriction appliquées, et les écarts de pratiques entre cadres nationaux ou équipes de modération locales.

K. Conteneurisation du module IA

Le module IA a été conteneurisé avec Docker [17]. La configuration sensible (identifiants PostgreSQL, clés API Gemini) est externalisée dans un fichier `.env` injecté à l'exécution via `-env-file`.

Listing 5 – Déploiement du conteneur IA

```
1 docker build -t shyamsubrun/megadonnee-dsa:latest .
2 sudo docker run -d \
3   --env-file /home/user/.env \
4   -p 8000:8000 \
5   --name megadonnee-dsa \
6   shyamsubrun/megadonnee-dsa:latest \
7   uvicorn api:app --host 0.0.0.0 --port 8000
```

VII. Cas d'Usage

Les cas d'usage suivants illustrent les capacités analytiques de la plateforme selon deux approches complémentaires : (1) une utilisation classique via l'interface graphique, (2) une utilisation assistée par intelligence artificielle via le module de requêtes en langage naturel.

A. CU1 : Analyse des disparités territoriales

Rôle : Chercheur en droit du numérique.

Question : Comment la politique de modération de TikTok diffère-t-elle entre la France et l'Allemagne ?

Approche via la plateforme :

1. Sélectionner la plateforme **TikTok**
2. Filtrer sur les pays **France** et **Allemagne**
3. Examiner la répartition territoriale via la carte européenne (Figure 16)
4. Examiner les statistiques détaillées au survol pour chaque pays, illustrées ici pour la France (Figure 17) et l'Allemagne (Figure 18)

Approche alternative via l'IA :

« Graphique comparatif France vs Allemagne pour TikTok : montre les actions de modération par pays avec un graphique en barres ou colonnes avec deux pays sur l'axe des catégories ou deux séries distinctes. »

Résultat : Identification des catégories de violation prédominantes par pays et des différences de traitement entre les deux États membres.



FIGURE 4 – Comparaison des actions de modération entre la France et l’Allemagne sur TikTok, générée via le module *Custom Query Chart*. *Perspective utilisateur (CU1 – Chercheur)* : sans aucune connaissance SQL, le chercheur obtient en moins d’une seconde une visualisation comparative répondant directement à sa question de recherche.

B. CU2 : Impact de l’automatisation

Rôle : Régulateur ou analyste de conformité.

Question : Quel est l’impact de l’automatisation sur la cohérence des décisions ?

Approche via la plateforme :

1. Consulter la section **Vue d’ensemble** (Figure 11) pour un état des lieux global
2. Examiner les indicateurs de détection et de décision automatisées
3. Analyser la répartition des sanctions selon les formats de contenu (Figures 19 et 20)

Approche alternative via l’IA :

« Analyse l’effet des décisions automatisées sur la cohérence opérationnelle : graphique comparant le délai moyen de traitement selon que la décision est automatisée ou non, ventilé par plateforme ou par catégorie. »



FIGURE 5 – Impact des décisions automatisées sur le délai moyen de traitement, générée via le module *Custom Query Chart*. *Perspective utilisateur (CU2 – Régulateur)* : cette visualisation permet de repérer les écarts de comportement entre plateformes et d’évaluer la place de l’automatisation dans les processus de modération.

Résultat : Visualisation des taux d’automatisation et analyse de la carte thermique croisée permettant d’identifier les plateformes et catégories où les décisions automatisées sont disproportionnées.

C. CU3 : Conformité légale

Rôle : Journaliste d’investigation.

Question : Quelles bases légales sont les plus fréquemment invoquées en France ?

Approche via la plateforme :

1. Filtrer sur le pays **France**
2. Accéder à la section **Bases juridiques** (Figure 15)
3. Classer les fondements juridiques par fréquence
4. Exporter le graphique en PNG pour rapport

Approche alternative via l’IA :

« Uniquement pour la France (FR) : classe les bases légales par fréquence et affiche un graphique en barres horizontales des 10 plus fréquentes. »

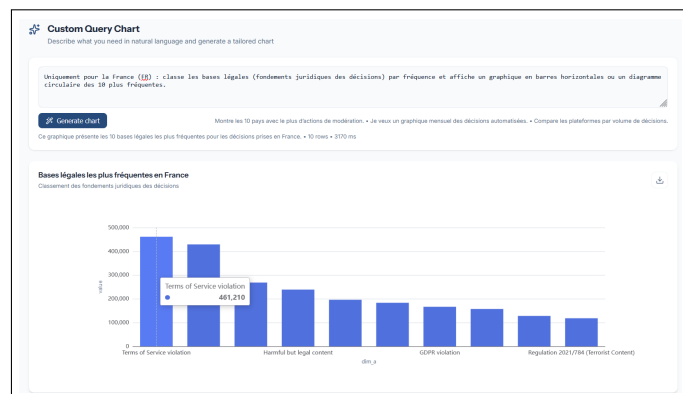


FIGURE 6 – Classement des bases légales les plus fréquentes en France, généré via le module *Custom Query Chart*. *Perspective utilisateur (CU3 – Journaliste)* : le journaliste identifie que les violations des conditions d’utilisation et les contenus illégaux au sens de l’article 3 du DSA figurent parmi les premiers motifs invoqués, puis exporte ce graphique pour son article.

Résultat : Classement des 10 motifs de décision les plus fréquents en France, prêt à être intégré dans un article ou rapport d’investigation.

D. CU4 : Analyse territoriale de la modération TikTok

Rôle : Analyste de conformité.

Question : Comment TikTok adapte-t-il ses décisions de modération selon les pays concernés ?

Approche via la plateforme :

1. Filtrer sur la plateforme **TikTok**
2. Visualiser la distribution globale via la section **Répartition géographique** (Figure 16)
3. Examiner les statistiques détaillées d’un pays au survol, par exemple la France (Figure 17)

4. Comparer les écarts territoriaux observés

Approche alternative via l'IA :

« Pour la plateforme TikTok uniquement, montre comment les décisions de modération varient selon le pays : graphique en carte thermique avec le pays en dimension principale et le type de décision ou le caractère automatisé en deuxième dimension. »

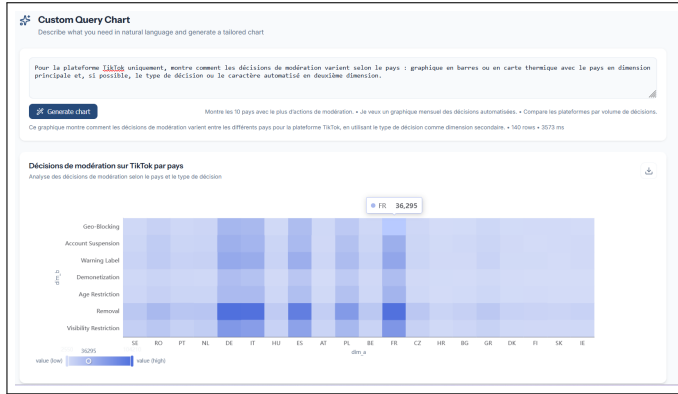


FIGURE 7 – Carte thermique des décisions de modération sur TikTok par pays et type de décision, générée via le module *Custom Query Chart*. *Perspective utilisateur (CU₄ – Analyste)* : cette vue met en évidence les territoires les plus actifs ainsi que les types de décisions dominants, renforçant la dimension comparative internationale de la plateforme.

Résultat : Visualisation et statistiques agrégées permettant d'identifier des différences de modération selon les territoires et d'en comprendre les logiques juridiques ou opérationnelles.

VIII. Analyse des Paramètres 3V des Mégadonnées et Performances

Les mégadonnées (*Big Data*) se caractérisent par les paramètres 3V, formalisés par Laney [25] : **Volume** (quantité massive de données à stocker et traiter), **Variété** (hétérogénéité des formats, structures et sources de données), et **Vitesse** (rapidité d'arrivée et de traitement des nouvelles données). Nous analysons ci-dessous comment ces trois dimensions se manifestent dans le cas de la base DSA transparency et comment notre plateforme y répond.

A. Volume

Les données DSA représentent environ 1 Go/jour compressé (environ 22 Go/jour décompressé). L'extrapolation mensuelle donne environ 660 Go par mois, et la base complète dépasse les 11 milliards d'entrées. Notre preuve de concept conserve une fenêtre filtrée d'environ 10 jours sur 15 plateformes, correspondant à environ 50 Go et plus de 17 millions d'entrées. **Notre projet satisfait ainsi au critère « grand volume » des mégadonnées.**

Extrapolation matérielle (scénario SMP). En extrapolant à un nœud SMP de type CRIANN (256 cœurs Intel Haswell à 2,2 GHz, 4 To de RAM), et en supposant un rendement pratique de parallélisation de 35 à 55 %, on obtient un gain réaliste de $\times 22$ à $\times 35$ par rapport à la MV actuelle (4 vCPU, 16 Go RAM). À architecture logicielle inchangée, notre système pourrait ainsi traiter entre 374 et 595 millions d'entrées sur une fenêtre d'exécution comparable.

B. Variété

La variété se manifeste à trois niveaux. **Structuel :** notre jeu de données comprend **27 colonnes** de types variés (8 types de données distincts : VARCHAR, TEXT, DATE, BOOLEAN, SMALLINT, INTEGER, JSONB, CHAR). **Sémantique :** **15 plateformes** aux conventions de nommage distinctes et plus de **50 catégories de violation** différentes. **Linguistique :** **24 langues** de contenu (UTF-8 multilingue : néerlandais, français, grec, allemand, maltais, etc.). On observe en outre la présence de valeurs manquantes (NULL, chaînes vides, ou NaN) dans environ 15–20 % des enregistrements selon les champs. Notre processus ETL normalise ces données vers un schéma relationnel optimisé. **Notre projet satisfait ainsi au critère « variété » des mégadonnées.**

C. Vitesse

La base DSA est mise à jour quotidiennement avec une fenêtre glissante de 6 mois. Nos performances d'insertion mesurées (environ 500 000 lignes par minute via \copy) permettent d'estimer qu'une ingestion quotidienne de 5 millions de nouvelles entrées prendrait environ 10 minutes sur la MV actuelle. Sur un nœud SMP de type CRIANN (256 cœurs), en parallélisant les insertions sur plusieurs partitions de table, ce délai pourrait être ramené à moins de 2 minutes. La mise en place d'un pipeline de streaming (Apache Kafka [28], AWS Kinesis) permettrait d'approcher une ingestion en quasi temps réel. **Notre projet satisfait ainsi au critère « vitesse » des mégadonnées dans le cadre d'un PoC académique.**

D. Environnement de production

La plateforme est déployée sur Google Cloud Platform :

- **Machine virtuelle Google Cloud** : série N2, 4 processeurs virtuels, 16 Go de mémoire vive
- **Stockage** : SSD 150 Go
- **PostgreSQL** : version 17
- **Accès** : serveur accessible via Internet (mandataire inverse Nginx)

Concernant le matériel physique sous-jacent, les machines virtuelles de série N2 sur GCP s'appuient sur des processeurs Intel Cascade Lake (3^e génération Xeon Scalable, jusqu'à 3,1 GHz en mode Turbo Boost). Les mesures de performance publiées par Google indiquent des débits réseau jusqu'à 32 Gbps et une bande passante disque SSD de l'ordre de 1 Go/s en lecture séquentielle.

E. Optimisations réalisées

1. **Pagination côté serveur** : limite par défaut à 1000 entrées par lecture
2. **Index composites** : accélération des requêtes filtrées
3. **Mémoire cache de TanStack Query** : évite les requêtes redondantes (5 minutes)

F. Motivations et périmètre des mesures

1) Évaluation des performances

Avant de présenter les résultats de performance, il convient d'expliquer pourquoi leur mesure est nécessaire dans le contexte d'un projet mégadonnées. À cette échelle, les performances d'un système ne peuvent pas être simplement postulées : elles doivent être mesurées, car un comportement acceptable sur un jeu de quelques milliers de lignes peut se dégrader de manière non linéaire à mesure que le volume croît.

Les sections VIII.A, VIII.B et VIII.C ont établi que notre jeu de données satisfait aux trois critères 3V : plus de 17 millions d'entrées pour 50 Go de données exploitables (Volume), 27 colonnes de types hétérogènes réparties sur 15 plateformes et 24 langues (Variété), et un flux entrant quotidien de plusieurs millions d'entrées côté source (Vitesse). Notre infrastructure, volontairement contrainte et représentative d'un déploiement académique réaliste, rend d'autant plus nécessaire la vérification que les optimisations mises en place – indexation, vues matérialisées, pagination, mémoire cache applicative – produisent effectivement les gains attendus.

Les mesures présentées dans cette section couvrent trois périmètres complémentaires :

- Les **métriques système** (processeur, mémoire vive, réseau, conteneurs), collectées en continu via notre infrastructure d'observabilité ;
- Les **temps de réponse des requêtes analytiques** sur la base PostgreSQL, en fonction du type de requête et du degré de parallélisme.

G. Infrastructure d'observabilité

1) Principe général

Afin de surveiller l'impact de nos services sur le serveur qui les héberge, nous avons mis en place une infrastructure d'observabilité s'appuyant sur des outils largement adoptés dans l'industrie informatique. Notre chaîne d'observabilité repose sur deux outils complémentaires : **Prometheus** pour la collecte des métriques, et **Grafana** pour leur visualisation. Prometheus collecte des métriques riches et fournit un langage de requête puissant, tandis que Grafana les transforme en visualisations exploitables [27].

2) Architecture de la chaîne d'observabilité

Notre chaîne d'observabilité distingue deux catégories de métriques. Les **métriques matérielles** sont exposées par **Node Exporter**, un agent Prometheus installé sur le

système hôte. Les **métriques logicielles** concernant les conteneurs Docker sont collectées par **cAdvisor**. L'ensemble est centralisé dans Prometheus puis restitué graphiquement dans Grafana.

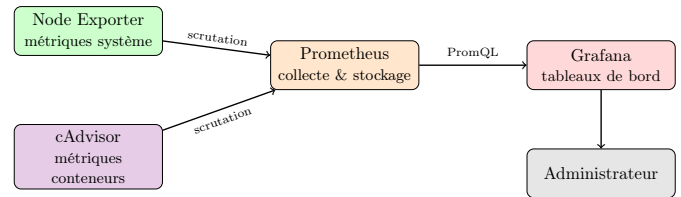


FIGURE 8 – Chaîne d'observabilité : collecte des métriques matérielles (Node Exporter) et logicielles (cAdvisor), agrégation dans Prometheus, et restitution graphique via Grafana.

3) Choix de la conteneurisation pour le déploiement

L'ensemble de la pile d'observabilité (Prometheus, Grafana, cAdvisor) est déployé sous forme de conteneurs Docker, orchestrés via Docker Compose. Ce choix évite tout conflit de dépendances avec les services applicatifs déjà en place, garantit une installation reproductible en une seule commande, et permet de retirer ou mettre à jour les outils de surveillance sans affecter la production.

Note sur InfluxDB. La configuration Docker Compose ci-dessous déclare également un service InfluxDB. Celui-ci a été intégré pour permettre une éventuelle persistance des métriques de séries temporelles à long terme (au-delà de la fenêtre de rétention par défaut de Prometheus de 15 jours). Dans le cadre de ce prototype, il n'est pas encore pleinement exploité mais constitue un axe d'évolution identifié.

Listing 6 – Extrait de configuration Docker Compose pour la pile d'observabilité

```
services:
  prometheus:
    image: prom/prometheus:v3.10.0
    container_name: prometheus
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
      - prometheus_data:/prometheus
    ports:
      - "9090:9090"
    restart: unless-stopped

  grafana:
    image: grafana/grafana:12.3.5
    container_name: grafana
    ports:
      - "3002:3000"
    volumes:
      - grafana_data:/var/lib/grafana
      - ./grafana/dashboards:/var/lib/grafana/dashboards:ro
      - ./grafana/provisioning/dashboards/dashboard.yml:/etc/grafana/provisioning/dashboards/dashboard.yml:ro
      - ./grafana/provisioning/datasources/datasource.yml:/etc/grafana/provisioning/datasources/datasource.yml:ro
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin
    depends_on:
      - prometheus
      - influxdb
    restart: unless-stopped
```

4) Tableau de bord Grafana – métriques système

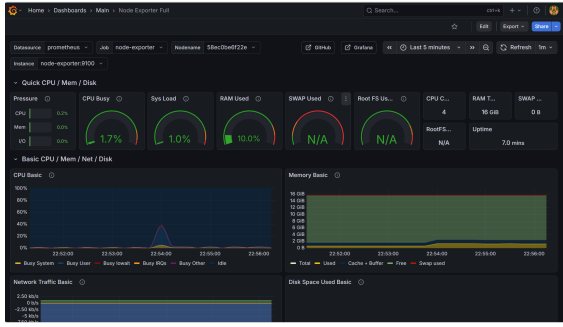


FIGURE 9 – Tableau de bord Grafana *Node Exporter Full* – métriques système en temps réel.

La section supérieure *Quick CPU / Mem / Disk* offre une lecture synthétique de l'état de la machine : processeur occupé à 1,7 %, charge système à 1,0, mémoire vive utilisée à 10,0 % (environ 1,6 Go sur les 16 Go disponibles), et espace d'échange non utilisé. La section *Basic CPU / Mem / Net / Disk* détaille ces métriques sous forme de séries temporelles : on observe un pic de processeur à environ 25 % lors du démarrage des services, suivi d'un retour immédiat à moins de 2 %.

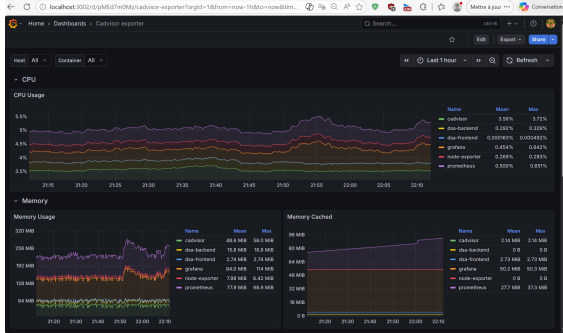


FIGURE 10 – Tableau de bord Grafana *cAdvisor* – métriques des conteneurs DSA en temps réel (processeur, mémoire, réseau par service).

Ce second tableau de bord, alimenté par *cAdvisor*, isole la consommation de chaque service conteneurisé (interface React, serveur Express, module FastAPI/IA) et permet d'identifier quel conteneur est responsable d'un pic de charge observé dans la vue système.

H. Temps de réponse des requêtes SQL

1) Protocole de mesure

Les mesures ont été réalisées directement sur la base PostgreSQL en production, hébergée sur la machine virtuelle N2 (4 vCPU, 16 Go RAM, SSD 150 Go), à l'aide de la commande `EXPLAIN ANALYZE`. Les mesures de performance portent sur les requêtes R1 et R3 ciblant **Facebook** (`platform_id = 54020`), la plateforme la plus représentée dans notre jeu de données avec environ 4,97 millions d'entrées. Quatre paliers ont été testés : 1M, 5M, 10M lignes, et la table complète à **17 millions** d'entrées.

2) Résultats par type de requête

Le Tableau 3 synthétise les temps d'exécution mesurés sur la table complète (17M entrées).

TABLE 3 – Temps d'exécution par type de requête (17M entrées, machine virtuelle N2)

Req.	Type	Exéc. (ms)
R1	Filtre simple	285,5
R2	Agrégation globale	1 466,9
R3	Filtre composite	1 680,2
R4	Jointure multi-tables	3 734,2

3) Scalabilité selon le volume de données

TABLE 4 – Évolution des temps d'exécution selon le volume (temps d'exécution en ms)

Volume	R1	R2	R3	R4
1M	21,8	90,7	158,0	228,7
5M	87,8	814,0	584,9	1 404,8
10M	169,1	1 626,7	1 097,7	2 566,9
17M	290,2	1 466,9	1 680,2	3 734,2

4) Impact du nombre de cœurs

TABLE 5 – Impact du parallélisme sur R2 (agrégation globale, 17M entrées)

Cœurs	Exéc. (ms)	Facteur
1	2 610,6	×1,00
2	1 461,9	×1,79
4	1 481,4	×1,76

Les résultats confirment un gain quasi linéaire entre 1 et 2 cœurs (facteur 1,79). En revanche, le passage de 2 à 4 cœurs n'apporte aucune amélioration supplémentaire : sur notre machine virtuelle à 4 vCPU avec un SSD partagé, la bande passante disque constitue vraisemblablement le goulot d'étranglement.

IX. Considérations RGPD

A. Données personnelles

Bien que les données DSA soient publiées dans un cadre de transparence réglementaire et ne contiennent pas d'identifiants utilisateurs directs, certains risques subsistent. Une réidentification indirecte peut, dans certains cas, devenir possible par croisement de dimensions géographiques, temporelles ou thématiques.

B. Risques liés à l'utilisation de l'IA générative

L'utilisation de l'API Gemini pour interpréter des requêtes en langage naturel introduit un point d'attention supplémentaire. Une partie du contenu saisi par l'utilisateur peut être transmise à un service externe exploité

par Google. Dans un contexte professionnel ou contentieux, une requête pourrait contenir des éléments permettant d'identifier une personne physique ou une situation sensible, constituant un transfert de données à caractère personnel soumis aux articles 44 à 49 du RGPD [4].

C. Mesures implémentées et leur effet

1. **Pas de stockage d'identifiants utilisateurs** : notre plateforme ne collecte ni ne conserve aucune donnée relative aux utilisateurs qui la consultent.
2. **Agrégation des données pour les visualisations** : toutes les visualisations présentent des statistiques agrégées et jamais des enregistrements individuels nominatifs.
3. **Accès contrôlé à la base de données** : la base PostgreSQL n'est pas directement exposée sur Internet ; seul le serveur applicatif y accède via un compte dédié aux privilèges limités.
4. **Externalisation des secrets** : mots de passe et clés API conservés hors du code source via `.env`.
5. **Minimisation des entrées envoyées au module IA** : seules les informations strictement nécessaires à l'interprétation de la requête sont transmises.

D. Limites et précautions d'usage

Dans sa forme actuelle, la plateforme doit être comprise comme une **preuve de concept académique**. Son usage est approprié pour l'exploration statistique de données publiques, mais il ne serait pas prudent de l'utiliser pour traiter des dossiers individuels, des situations contentieuses ou des données contenant des informations personnelles.

Figures et captures d'écran de la plateforme

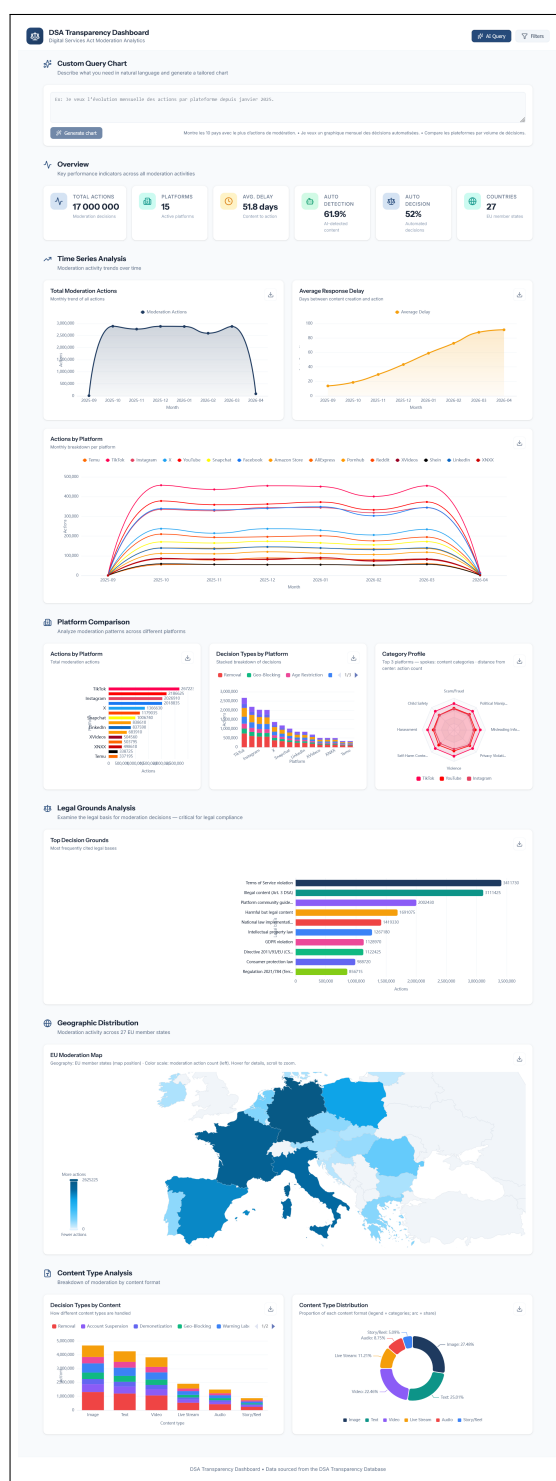


FIGURE 11 – Vue d'ensemble complète du tableau de bord de la DSA Transparency – de haut en bas : module *Custom Query Chart*, Vue d'ensemble (17 000 000 actions totales, 15 plateformes, délai moyen 51,8 jours, détection automatisée 61,9 %, décision automatisée 52 %, 27 pays), Séries temporelles, Évolution mensuelle par plateforme, Comparaison des plateformes, Bases juridiques, Répartition géographique, et Type de contenu. *Perspective utilisateur* : cette capture illustre la cohérence visuelle de l'interface et la richesse fonctionnelle accessible depuis un seul écran de navigation.

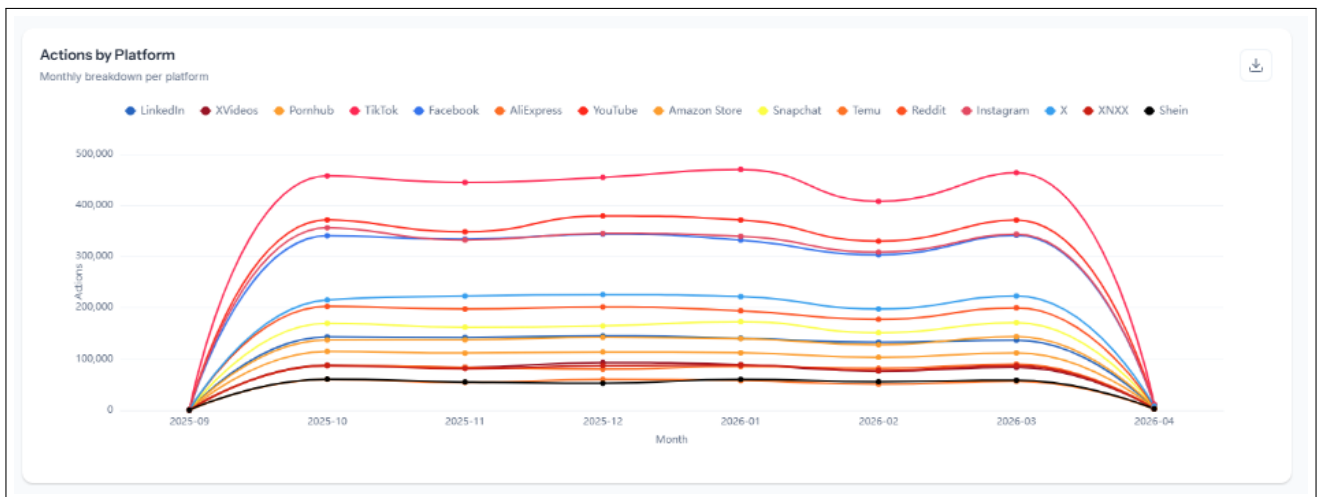


FIGURE 12 – Évolution mensuelle des actions de modération par plateforme de septembre 2025 à avril 2026, pour les 15 plateformes couvertes : LinkedIn, XVideos, Pornhub, TikTok, Facebook, AliExpress, YouTube, Amazon Store, Snapchat, Temu, Reddit, Instagram, X, XNXX et Shein. Les courbes montrent une activité soutenue entre octobre 2025 et mars 2026, avec un pic observable pour plusieurs plateformes entre octobre et décembre 2025, avant un ralentissement en avril 2026 correspondant à la fin de la fenêtre glissante de collecte. *Perspective utilisateur (CU1 – Chercheur)* : après avoir sélectionné une plateforme spécifique dans les filtres, le chercheur isole sa courbe pour observer les pics d'activité mensuelle et les corrélérer avec des événements réglementaires ou médiatiques.

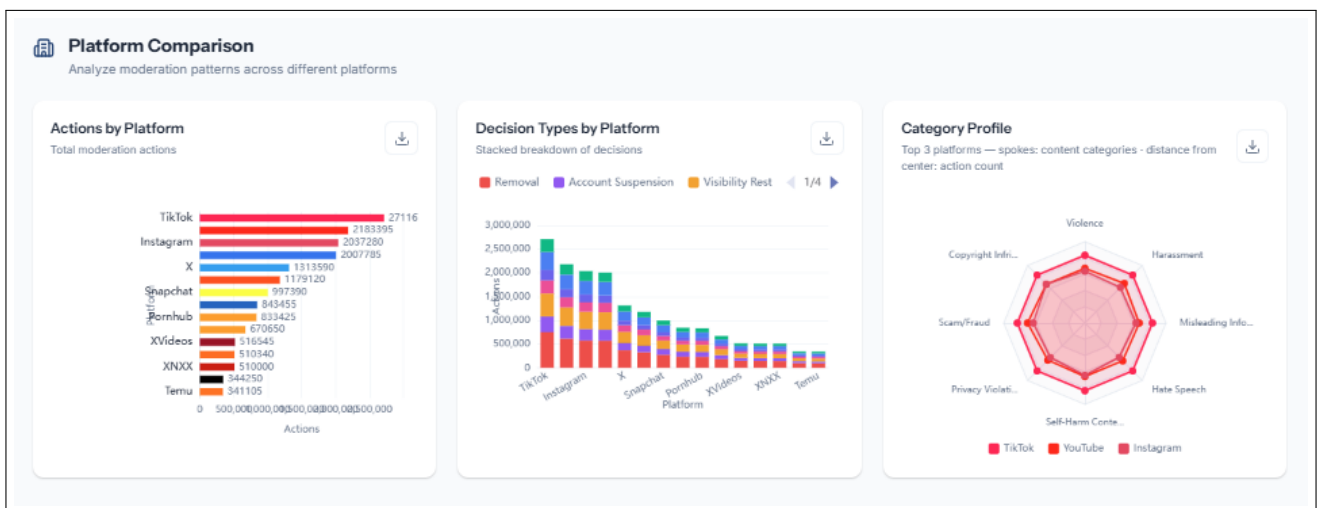


FIGURE 13 – Section Comparaison des plateformes – (à gauche) Volume total d'actions par plateforme en histogramme horizontal : TikTok arrive en tête, suivi d'Instagram, de X, de Snapchat, puis de Pornhub, XVideos, XNXX et Temu ; (au centre) répartition des types de décision empilés par plateforme (suppression, suspension de compte, restriction de visibilité, etc.) ; (à droite) diagramme radar du profil catégoriel des trois premières plateformes – TikTok, YouTube et Instagram – sur les dimensions Violence, Harcèlement, Information trompeuse, Discours haineux, Contenu auto-mutilation, Violation de la vie privée, Fraude/Arnaque et Contrefaçon. *Perspective utilisateur (CU1 – Chercheur)* : la vue radar permet de comparer instantanément si TikTok, YouTube et Instagram présentent des profils de catégories similaires ou distincts, renseignant sur leurs politiques de modération respectives.

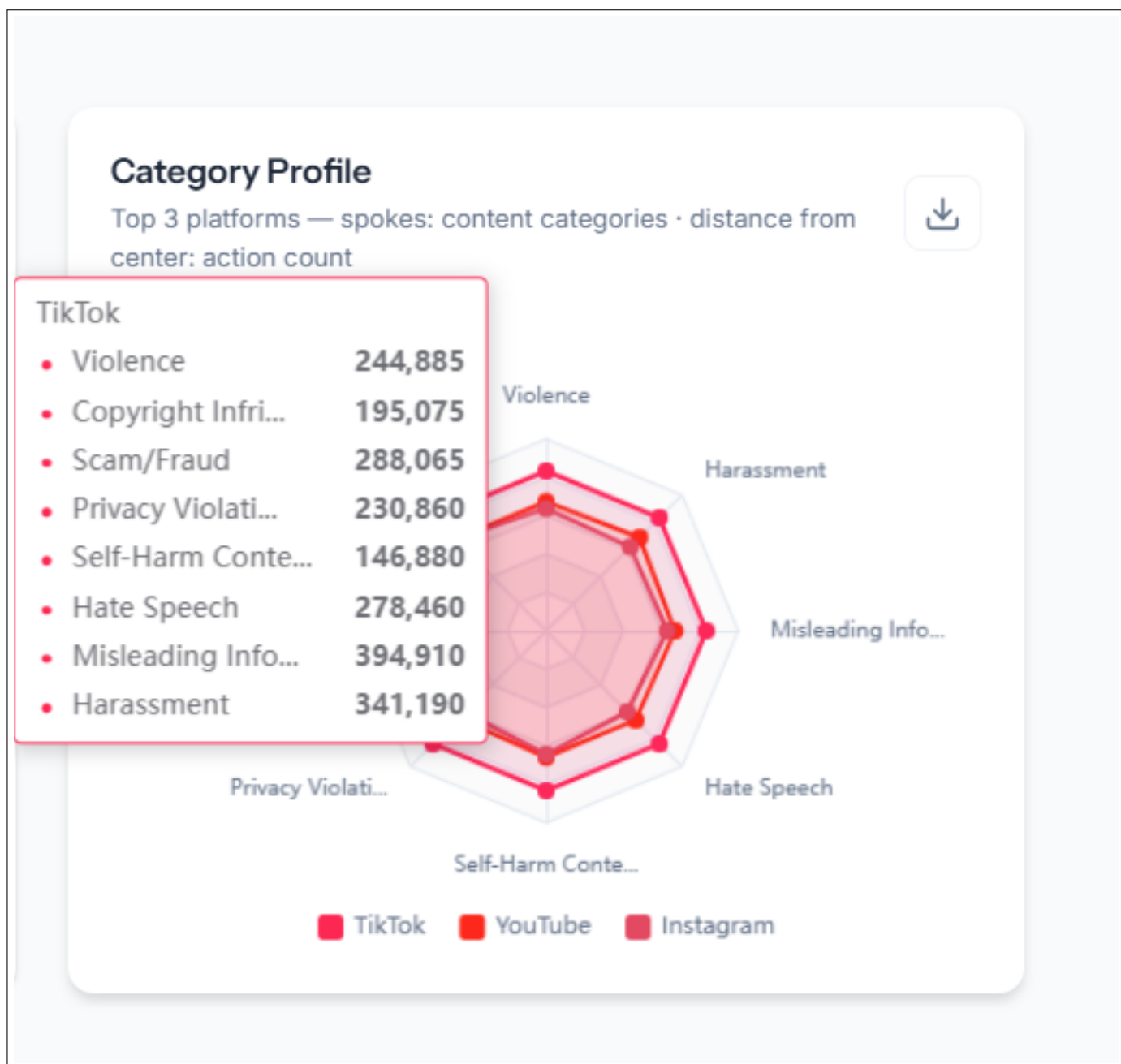


FIGURE 14 – Profil catégoriel des plateformes sous forme de diagramme radar. Chaque axe représente une catégorie de contenu (violence, harcèlement, désinformation, discours haineux, fraude, etc.), et chaque polygone correspond à une plateforme (TikTok, YouTube, Instagram). L'interaction par survol permet d'obtenir les valeurs exactes pour chaque catégorie, par exemple pour TikTok : 394 910 cas de désinformation, 341 190 de harcèlement et 288 065 de fraude. *Perspective utilisateur (CU1 – Chercheur)* : cette visualisation permet d'identifier rapidement les spécificités de chaque plateforme et de comparer leurs profils de modération, en mettant en évidence les catégories dominantes et les différences structurelles entre acteurs.



FIGURE 15 – Section Bases juridiques – Classement des 10 motifs de décision les plus fréquemment cités, par ordre décroissant : *Terms of Service violation* (3 391 330), *Illegal content (Art. 3 DSA)* (3 115 335), *Platform community guidelines* (1 975 655), *Harmful but legal content* (1 685 975), *National law implementation* (1 409 810), *Intellectual property law* (1 291 320), *Directive 2011/93/EU (CSAM)* (1 148 520), *GDPR violation* (1 139 085), *Consumer protection law* (990 930) et *Regulation 2021/784 (Terrorism)* (852 040). *Perspective utilisateur (CU3 – Journaliste)* : après filtrage sur la France, le journaliste identifie les motifs prédominants et exporte ce graphique pour l'intégrer dans son article ou rapport d'investigation.

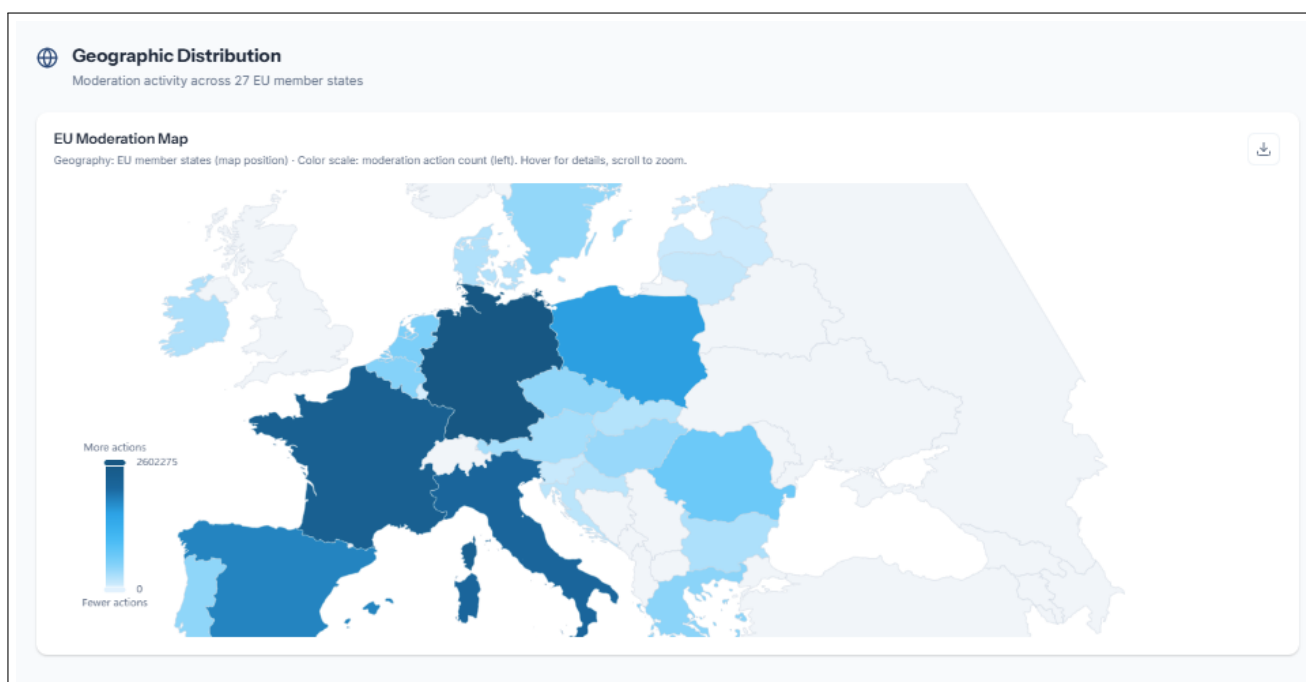


FIGURE 16 – Carte choroplèthe interactive de l'UE – Distribution territoriale des actions de modération dans les 27 États membres. L'intensité de la couleur bleue reflète le volume d'actions par pays selon une échelle de 0 à 2 502 275 actions. La France et l'Allemagne apparaissent en bleu très foncé, attestant du volume d'actions le plus élevé ; la Pologne et d'autres États de l'Europe centrale présentent des teintes intermédiaires, tandis que les États baltes et nordiques restent en teintes claires. *Perspective utilisateur (CU4 – Analyste)* : l'analyste survole la carte pour identifier rapidement les pays à fort volume d'activité de modération, puis filtre sur un État membre spécifique pour afficher les statistiques détaillées (taux d'automatisation, délai moyen, catégorie dominante, plateforme principale).

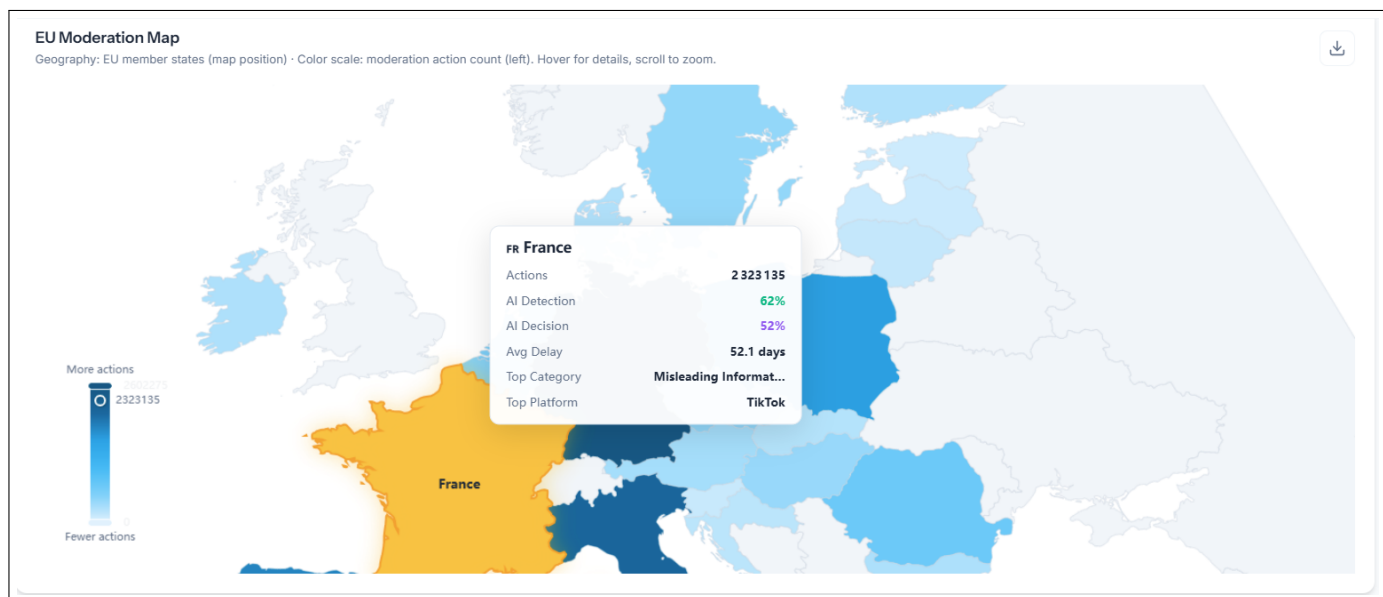


FIGURE 17 – Carte choroplèthe interactive de l'Union européenne avec focus sur la France. Lors du survol, une infobulle affiche des indicateurs détaillés : nombre total d'actions de modération, taux de détection automatisée, taux de décision automatisée, délai moyen de traitement, catégorie principale et plateforme dominante. *Perspective utilisateur (CU1 – Chercheur)* : cette visualisation permet d'obtenir une analyse détaillée pour la France, facilitant la comparaison avec l'Allemagne dans une analyse transnationale.

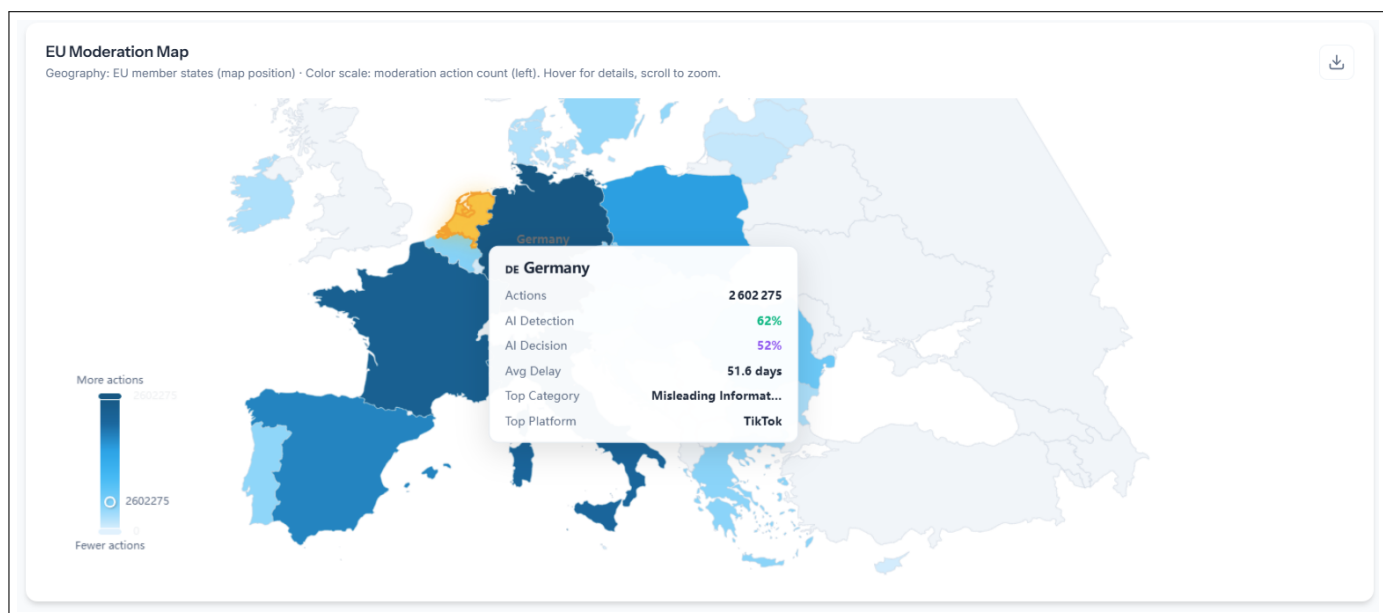


FIGURE 18 – Carte choroplèthe avec focus sur l'Allemagne. Lors du survol, une infobulle affiche des indicateurs détaillés : nombre total d'actions de modération, taux de détection automatisée, taux de décision automatisée, délai moyen de traitement, catégorie principale et plateforme dominante. *Perspective utilisateur (CU1 – Chercheur)* : cette visualisation permet d'obtenir une analyse détaillée pour l'Allemagne, facilitant la comparaison avec la France dans une analyse transnationale.

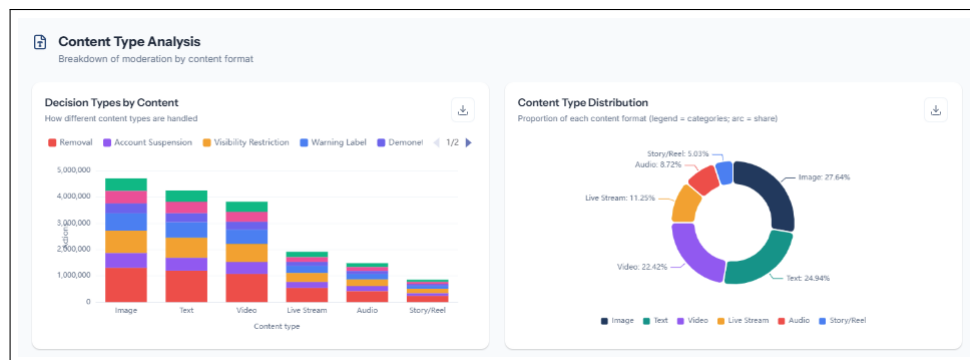


FIGURE 19 – Section Type de contenu – (à gauche) Histogramme empilé des types de décision (suppression, suspension de compte, restriction de visibilité, avertissement, démonétisation, et autres) ventilés par format de contenu : image, texte, vidéo, diffusion en direct (*live stream*), audio et story/reel. Les images et les textes concentrent les plus gros volumes d’actions, de l’ordre de 4 à 5 millions chacun, avec une large majorité de suppressions et de restrictions de visibilité. (à droite) Diagramme en anneau de la distribution globale des formats : image (27,64 %), texte (24,94 %), vidéo (22,42 %), diffusion en direct (11,25 %), audio (8,72 %) et story/reel (5,03 %). *Perspective utilisateur (CU2 – Régulateur)* : cette double vue permet d’analyser si certains formats concentrent davantage certaines sanctions, et d’identifier si des formats émergents tels que la diffusion en direct ou les stories font l’objet d’un traitement réglementaire spécifique.



FIGURE 20 – Analyse des types de décision par format de contenu. Chaque barre empilée représente la répartition des actions de modération (suppression, suspension de compte, restriction de visibilité, avertissement, démonétisation, restriction d'âge, blocage géographique) selon le type de contenu (image, texte, vidéo, diffusion en direct, audio, story). L'interaction par survol permet d'afficher les valeurs exactes, par exemple pour les images : 1 305 005 suppressions, 565 845 suspensions et 844 305 restrictions de visibilité. *Perspective utilisateur (CU2 – Régulateur)* : cette visualisation met en évidence les différences de traitement selon le format de contenu, révélant que les images et vidéos concentrent la majorité des actions et présentent une plus grande diversité de sanctions.

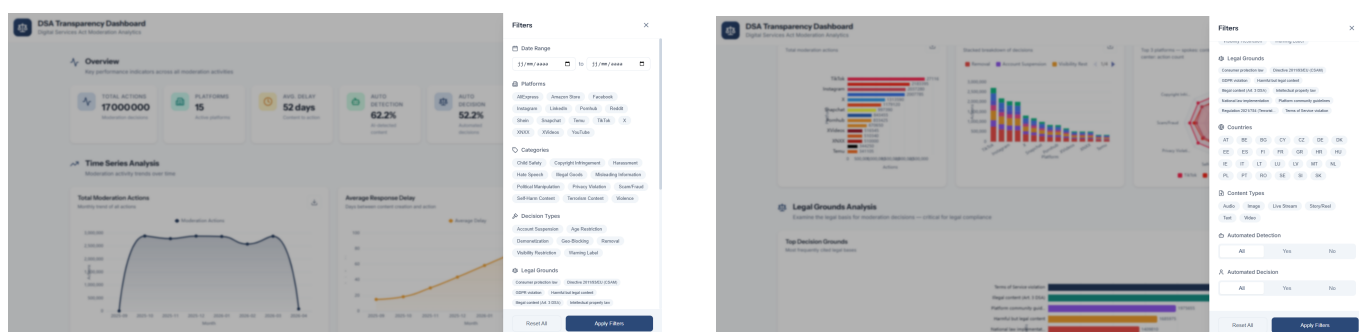


FIGURE 21 – Panneau de filtres multi-critères – À gauche : plage de dates, plateformes, catégories, types de décision et bases juridiques. À droite : pays (27 codes ISO), types de contenu, détection automatisée et décision automatisée avec boutons de réinitialisation et d’application. *Perspective utilisateur* : ce panneau est le point d’entrée principal de tous les cas d’usage – l’utilisateur configure ici ses critères avant de naviguer entre les sections d’analyse.



FIGURE 22 – Interface *Custom Query Chart* – Module de génération automatique de requêtes SQL depuis le langage naturel. L'utilisateur décrit sa requête en français ; des exemples prédéfinis (« Montre les 10 pays avec le plus d'actions de modération », « Compare les plateformes par volume de décisions ») guident les utilisateurs novices. *Perspective utilisateur (tous profils)* : ce module permet à tout utilisateur – chercheur, journaliste, régulateur ou analyste – sans aucune connaissance SQL d'explorer les données de façon personnalisée, au-delà des vues prédéfinies du tableau de bord.



FIGURE 23 – Interface native de cAdvisor – Vue d'ensemble des conteneurs et des ressources système. Cette interface expose directement les conteneurs Docker et Podman ainsi que leur organisation en sous-conteneurs (cgroups), et fournit des métriques brutes telles que l'allocation CPU, la mémoire disponible et l'utilisation en temps réel.

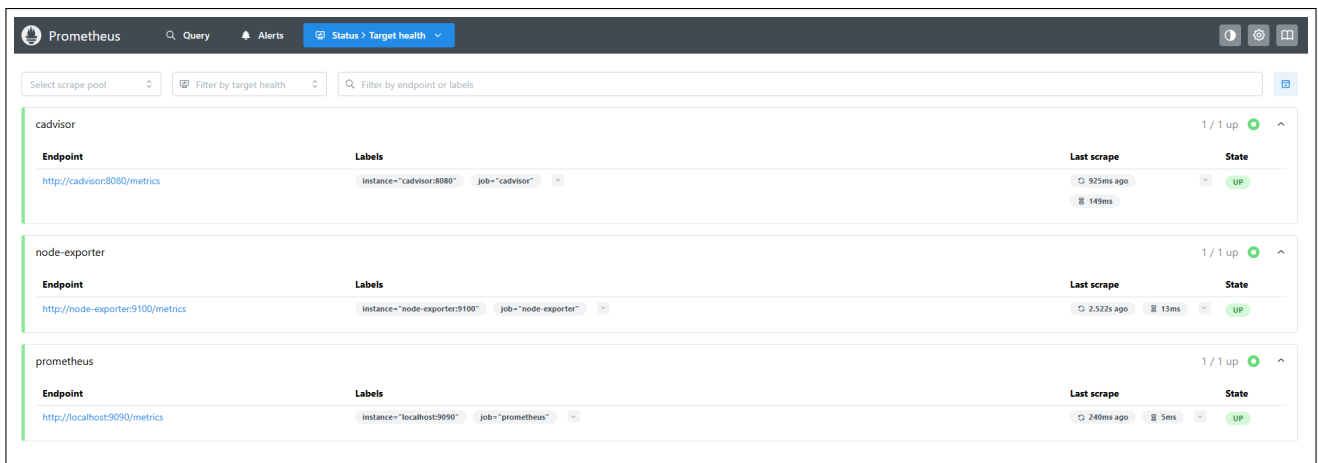


FIGURE 24 – Interface Prometheus – État des cibles de collecte des métriques. Cette vue présente les différents services supervisés par Prometheus, notamment cAdvisor, node-exporter et Prometheus lui-même. Chaque cible expose un endpoint `/metrics` interrogé périodiquement, dont l'état (*UP*) confirme la bonne collecte des données.

I. Référence de l'API

A. Points d'accès de consultation

TABLE 6 – Points d'accès GET de l'API REST

Méthode	Point d'accès	Description
GET	/api/moderation	Liste paginée des entrées avec filtres
GET	/api/moderation/stats	Statistiques agrégées
GET	/api/filters	Options de filtres disponibles
GET	/api/verification	Rapport de qualité des données
GET	/health	Vérification de l'état du service

B. Points d'accès TALN et administration

Les points d'accès suivants ont été développés côté serveur applicatif pour les fonctionnalités avancées d'IA et d'administration. Ces API sont fonctionnelles mais ne sont pas encore exposées dans l'interface de la couche présentation.

Méth.	Point d'accès	Description
POST	/nlp/decision	Analyse une phrase via Gemini, reconstruit les caractéristiques et prédit la décision. Entrée : <code>text</code> . Retour : <code>raw_output</code> .
POST	/nlp/sql	Génère du SQL depuis le langage naturel. Entrée : <code>text</code> . Retour : <code>sql</code> .
POST	/admin/ref-values	Régénère le JSON des valeurs de référence DSA.
POST	/admin/prompt	Régénère l'invite depuis les attributs DSA.

TABLE 7 – Points d'accès TALN et administration (serveur applicatif)

II. Conclusion

Ce travail a été réalisé dans le cadre du cours mégadonnées du diplôme de Master 2 Logiciels Sûrs de l'Université Paris-Est Créteil (UPEC), promotion 2025-2026. Il a permis de développer une plateforme complète d'analyse à interface et visualisation graphiques de la base de données de la DSA transparency. La plateforme offre une interface intuitive ne nécessitant aucune programmation, avec huit sections d'analyse interactive, un filtrage multicritères dynamique et des visualisations exportables. Un module innovant à base d'apprentissage automatique permet de prédire le type de décision de modération avec une exactitude de 97 %, et un module de TALN permet de formuler des requêtes analytiques en langage naturel sans aucune connaissance en SQL.

En termes de performances mesurées, la plateforme traite plus de 17 millions d'entrées sur une MV N2 (4 vCPU, 16 Go RAM, SSD 150 Go) avec des temps de réponse inférieurs à 2 secondes pour les requêtes agrégées filtrées. L'ingestion initiale du jeu de données (50 Go, 17 millions de lignes) a été réalisée en moins de 3 heures via l'import

natif PostgreSQL `\copy`. Ces mesures confirment l'adéquation de notre architecture pour les volumes DSA d'une fenêtre glissante de 10 jours.

Parmi les limitations subsistantes : la fenêtre de données est limitée à 6 mois en raison d'une contrainte de l'API, certaines classes rares restent difficiles à prédire, et la carte géographique réelle n'a pas été intégrée faute de données GeoJSON.

III. Perspectives

Plusieurs axes d'amélioration ont été identifiés : intégration des exports CSV et PDF, expérimentation de modèles plus avancés (réseaux de neurones profonds, Transformers), amélioration de la gestion du déséquilibre des classes, mise en place d'une validation croisée, extension du module TALN pour les requêtes multi-intentions, mise en place d'un pipeline de traitement en quasi temps réel via Apache Kafka [28] (environ 5 millions d'entrées par jour), et renforcement de la conception IHM selon les heuristiques de Nielsen [29], les principes ISO 9241-110 [30] et les travaux de Shneiderman sur la visualisation orientée tâches [31].

Remerciements

Nous tenons à remercier chaleureusement M. Gaétan Hains et M. Pierre Valarcher, nos encadrants pédagogiques, pour leur direction scientifique, leurs conseils, leur disponibilité et leurs retours constructifs tout au long de ce projet.

Références

- [1] Règlement (UE) 2022/2065 du Parlement européen et du Conseil du 19 octobre 2022 relatif à un marché unique des services numériques (Digital Services Act).
- [2] Commission Européenne, "DSA Transparency Database," [En ligne]. Disponible : <https://transparency.dsa.ec.europa.eu/>
- [3] Directive 2011/93/UE du Parlement européen et du Conseil du 13 décembre 2011 relative à la lutte contre les abus sexuels et l'exploitation sexuelle des enfants, ainsi que la pédopornographie (CSAM).
- [4] Règlement (UE) 2016/679 du Parlement européen et du Conseil du 27 avril 2016 relatif à la protection des personnes physiques à l'égard du traitement des données à caractère personnel (RGPD).
- [5] Organisation internationale de normalisation, "ISO 3166-1 :2020 – Codes pour la représentation des noms de pays et de leurs subdivisions," [En ligne]. Disponible : <https://www.iso.org/iso-3166-country-codes.html>
- [6] Organisation internationale de normalisation, "ISO 639-1 :2002 – Codes pour la représentation des noms de langues," [En ligne]. Disponible : <https://www.iso.org/iso-639-language-code>

- [7] P. Leach, M. Mealling, R. Salz, “A Universally Unique Identifier (UUID) URN Namespace,” RFC 4122, IETF, juillet 2005. [En ligne]. Disponible : <https://tools.ietf.org/html/rfc4122>
- [8] Meta, “React – Bibliothèque JavaScript pour la construction d’interfaces utilisateur,” [En ligne]. Disponible : <https://react.dev/>
- [9] Apache Software Foundation, “Apache ECharts,” [En ligne]. Disponible : <https://echarts.apache.org/>
- [10] Prisma, “ORM nouvelle génération pour Node.js et TypeScript,” [En ligne]. Disponible : <https://www.prisma.io/>
- [11] PostgreSQL Global Development Group, “PostgreSQL : The World’s Most Advanced Open Source Relational Database,” [En ligne]. Disponible : <https://www.postgresql.org/>
- [12] F. Pedregosa *et al.*, “Scikit-learn : Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [13] E. You, “Vite – Outillage nouvelle génération pour le développement web,” [En ligne]. Disponible : <https://vitejs.dev/>
- [14] T. Linsley, “TanStack Query – Gestion asynchrone d’état,” [En ligne]. Disponible : <https://tanstack.com/query/>
- [15] Google, “Documentation de l’API Gemini,” [En ligne]. Disponible : <https://ai.google.dev/docs>
- [16] S. Ramírez, “FastAPI – Cadriciel web moderne et rapide pour Python,” [En ligne]. Disponible : <https://fastapi.tiangolo.com/>
- [17] Docker Inc., “Documentation Docker,” [En ligne]. Disponible : <https://docs.docker.com/>
- [18] OpenJS Foundation, “Node.js – Environnement d’exécution JavaScript,” [En ligne]. Disponible : <https://nodejs.org/>
- [19] OpenJS Foundation, “Express.js – Cadriciel web minimaliste pour Node.js,” [En ligne]. Disponible : <https://expressjs.com/>
- [20] Oracle Corporation, “MySQL – Système de gestion de base de données relationnelle,” [En ligne]. Disponible : <https://www.mysql.com/>
- [21] D. R. Hipp, “SQLite – Base de données embarquée,” [En ligne]. Disponible : <https://www.sqlite.org/>
- [22] Amazon Web Services, “Documentation AWS,” [En ligne]. Disponible : <https://docs.aws.amazon.com/>
- [23] Microsoft, “Documentation Microsoft Azure,” [En ligne]. Disponible : <https://docs.microsoft.com/azure/>
- [24] Google, “Documentation Google Cloud Platform,” [En ligne]. Disponible : <https://cloud.google.com/docs>
- [25] D. Laney, “3D Data Management : Controlling Data Volume, Velocity and Variety,” META Group Research Note, 2001. [En ligne]. Disponible : <https://blogs.gartner.com/doug-laney/files/2012/01/ad949-3D-Data-Management-Controlling-Data-Volume-Velocity-and-Variety.pdf>
- [26] S. Juba, A. Vannahme, “Learning PostgreSQL,” Packt Publishing, 2015.
- [27] OpsRamp, “Prometheus vs Grafana : Key Differences and Use Cases,” [En ligne]. Disponible : <https://www.opsramp.com/guides/prometheus-monitoring/prometheus-vs-grafana/>
- [28] Apache Software Foundation, “Apache Kafka – Distributed Event Streaming Platform,” [En ligne]. Disponible : <https://kafka.apache.org/>
- [29] J. Nielsen, “10 Usability Heuristics for User Interface Design,” Nielsen Norman Group, 1994.
- [30] ISO, “ISO 9241-110 :2020 – Ergonomics of human-system interaction – Interaction principles,” 2020.
- [31] B. Shneiderman, “The eyes have it : A task by data type taxonomy for information visualizations,” IEEE Symposium on Visual Languages, 1996.